Lecture 3: Gradient Descent

Course Teacher: Md. Shariful Islam Bhuyan

Our Learning Journey Today

1. Gradient Descent: The Iterative Approach
2. Hands-On: Implementation from Scratch
3. Generalization: The Real Goal
4. Weight Decay: Fighting Overfitting
5. Summary: What We've Learned

The Gradient Descent Intuition

★ Think About It

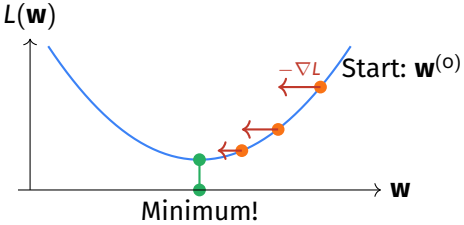
Imagine you're blindfolded on a hilly landscape. Your goal: find the lowest point. What would you do?

The Gradient Descent Intuition

★ Think About It

Imagine you're blindfolded on a hilly landscape. Your goal: find the lowest point. What would you do?

Natural strategy: Feel which way is downhill, take a step that direction, repeat!



The gradient ∇L points in the direction of steepest *ascent*.
The negative gradient $-\nabla L$ points *downhill*!

The Gradient Descent Update Rule

Gradient Descent Update

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

Key components:

- $\mathbf{w}^{(t)}$: Current parameters at iteration t
- $\nabla_{\mathbf{w}} L$: Gradient (direction of steepest ascent)
- η: Learning rate (step size)—crucial hyperparameter!
- $-$: Go *opposite* to gradient (downhill!)

Why does this work?
For small enough η, taking a step in the direction $-\nabla L$ is **guaranteed** to decrease the loss (for smooth functions).

The Three Flavors of Gradient Descent

Batch GD

Use **all** n **examples** to compute gradient.

✓ Exact gradient
✓ Stable updates
× Very slow for large n
× Memory intensive

Stochastic GD (SGD)

Use **1 random example** per update.

✓ Very fast updates
✓ Can escape local minima
× Noisy gradient
× Erratic convergence

Minibatch SGD

Use $|B|$ **examples** (32–256).

✓ Best of both worlds!
✓ GPU-friendly
✓ Reasonably stable
✓ Standard choice

♣ Key Insight

Minibatch SGD is the workhorse of deep learning. It balances computational efficiency with gradient quality, and leverages GPU parallelism beautifully!

Minibatch SGD: The Algorithm

Minibatch SGD Update for Linear Regression

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|B|} \sum_{i \in B_t} \nabla_{(\mathbf{w},b)} \ell^{(i)}(\mathbf{w}, b)$$

Expanded for linear regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|B|} \sum_{i \in B_t} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)$$

$$b \leftarrow b - \frac{\eta}{|B|} \sum_{i \in B_t} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)$$

The training loop:

1. Initialize $\mathbf{w}$ randomly, $b = 0$
2. For each epoch (pass through data):
   - Shuffle the training data
   - For each minibatch B_t:
     - Compute gradient of B_t
     - Update parameters

The Learning Rate: A Critical Hyperparameter

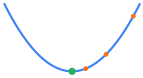
Too small:

Painfully slow!

Too large:

Oscillates/diverges!

Just right:



Converges nicely!

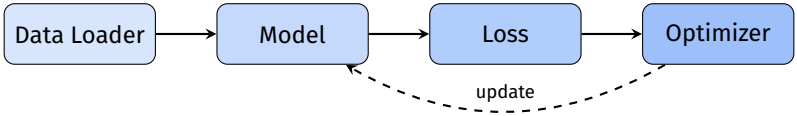
Practical advice:

- Start with $\eta = 0.01$ or 0.001
- Monitor the loss curve
- Use learning rate schedules

△ Watch Out

The learning rate is often the **most important** hyperparameter. Getting it wrong can mean the difference between success and failure!

The Training Pipeline



This pattern repeats for *every* deep learning model!

```
for epoch in range(num_epochs):
    for X_batch, y_batch in data_loader:
        # Forward pass
        y_hat = model(X_batch)
        loss = loss_fn(y_hat, y_batch)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Model Definition

Linear Regression Model

```
class LinearRegressionScratch:
    def __init__(self, num_inputs, lr, sigma=0.01):
        # Initialize weights from N(0, sigma^2)
        self.w = torch.normal(0, sigma, (num_inputs, 1),
                              requires_grad=True)
        self.b = torch.zeros(1, requires_grad=True)
        self.lr = lr

    def forward(self, X):
        """Compute predictions: y_hat = Xw + b"""
        return torch.matmul(X, self.w) + self.b

    def loss(self, y_hat, y):
        """Mean squared error loss"""
        return ((y_hat - y.reshape(y_hat.shape)) ** 2 / 2).mean()
```

♣ Key Insight

`requires_grad=True` tells PyTorch to track operations for automatic differentiation!

SGD Optimizer

Minibatch SGD Optimizer

```
class SGD:
    def __init__(self, params, lr):
        self.params = params
        self.lr = lr

    def step(self):
        """Update parameters using gradients"""
        for param in self.params:
            # w <- w - lr * grad
            param.data -= self.lr * param.grad

    def zero_grad(self):
        """Reset gradients to zero"""
        for param in self.params:
            if param.grad is not None:
                param.grad.zero_()
```

Why `param.data`**?** Modifying `.data` updates the tensor without creating new computation graph nodes.
Why `zero_grad()`**?** PyTorch *accumulates* gradients by default!

The Complete Training Loop

Training Loop

```
def train(model, data_loader, num_epochs):
    optimizer = SGD([model.w, model.b], model.lr)
    for epoch in range(num_epochs):
        for X_batch, y_batch in data_loader:
            # Forward pass
            y_hat = model.forward(X_batch)
            l = model.loss(y_hat, y_batch)
            # Backward pass
            optimizer.zero_grad()
            l.backward()  # Compute gradients via autograd
            optimizer.step()  # Update parameters
        # Log progress
        print(f'Epoch {epoch+1}, Loss: {l:.4f}')
```

♣ Key Insight

`l.backward()` is the magic! PyTorch automatically computes gradients for all tensors with `requires_grad=True`.

Why Vectorization Matters

★ Think About It

Why do we use `torch.matmul` instead of Python loops?

Experiment: Add two 10,000-dimensional vectors.

Python Loop

```
c = torch.zeros(n)
for i in range(n):
    c[i] = a[i] + b[i]
# Time: ~0.18 seconds
```

Vectorized

```
c = a + b
# Time: ~0.0004 seconds
```

That's a 450× speedup! Why?

- Python loops: interpreter overhead, type checking each iteration
- Vectorized ops: optimized C/Fortran libraries (BLAS/LAPACK)
- GPU: thousands of parallel operations simultaneously

Training Error vs. Generalization Error

★ Think About It

If our model perfectly memorizes the training data, have we succeeded?

Training Error vs. Generalization Error

★ Think About It

If our model perfectly memorizes the training data, have we succeeded?

No! We care about performance on *new, unseen data*!

Training Error R_{emp}
What we *measure*:

$$R_{\text{emp}} = \frac{1}{n} \sum_{i=1}^{n} \ell(\mathbf{x}^{(i)}, y^{(i)}, f)$$

Average loss on training data.

Generalization Error R
What we *care about*:

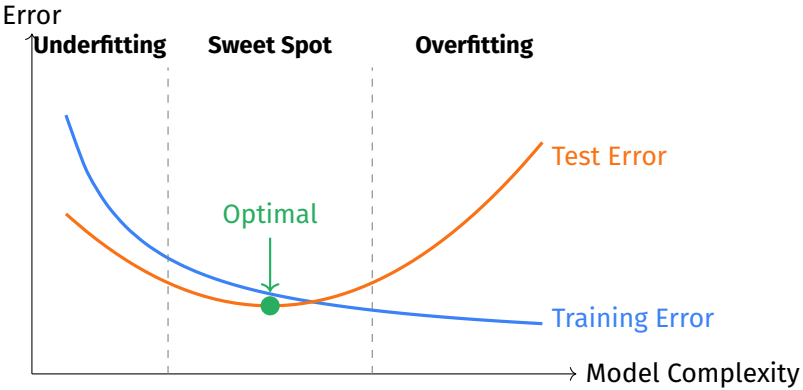
$$R = \mathbb{E}_{(\mathbf{x},y)\sim P}[\ell(\mathbf{x}, y, f)]$$

Expected loss on *any* data from the distribution.

The Generalization Gap

We can only *measure* training error, but we *want* low generalization gap = $R_{\text{emp}} - R$

Underfitting vs. Overfitting



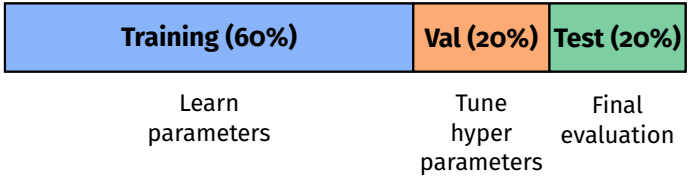
Underfitting:

- Model too simple
- High training & test error

Overfitting:

- Model too complex
- Low training, high test error

Model Selection: Train / Validation / Test Split



The Golden Rules

1. **Never** use test data for model selection!
2. Use validation data to choose hyperparameters
3. Touch test data **only once**—for final reporting

K-fold cross-validation: When data is scarce, split training data into K parts, rotate which part is validation, average results.

The Regularization Idea

★ Think About It

We're overfitting. We can't get more data. What else can we do?

The Regularization Idea

★ Think About It

We're overfitting. We can't get more data. What else can we do?

Key insight: Among all functions, $f = 0$ is the "simplest."
$\Rightarrow$ Penalize large weights!

ℓ_2 Regularized Loss (Weight Decay)

$$L_{\text{reg}}(\mathbf{w}, b) = L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

What this does:

- $\lambda = 0$: No regularization (original loss)
- $\lambda > 0$: Large weights incur penalty
- Larger λ: Stronger preference for small weights

Why "Weight Decay"?

Look at the gradient update:

$$\nabla_{\mathbf{w}} L_{\text{reg}} = \nabla_{\mathbf{w}} L + \lambda \mathbf{w}$$

SGD update becomes:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \eta(\nabla_{\mathbf{w}} L + \lambda \mathbf{w}) \\ &= \mathbf{w} - \eta \nabla_{\mathbf{w}} L - \eta \lambda \mathbf{w} \\ &= (1 - \eta\lambda)\mathbf{w} - \eta \nabla_{\mathbf{w}} L \end{aligned}$$

♣ Key Insight

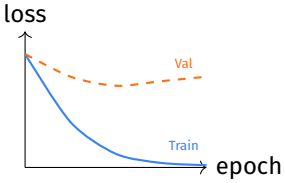
At each step, weights are **multiplied by** $(1 - \eta\lambda) < 1$!
They literally "decay" toward zero. Hence the name: weight decay.

Effect: No single weight can grow too large; model must distribute importance across features.

Weight Decay in Action

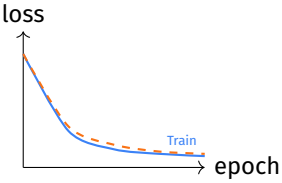
Experiment: 200 features, only 20 training examples (severe overfitting risk!)

Without regularization ($\lambda = 0$):



Huge gap = Overfitting!

With regularization ($\lambda = 3$):

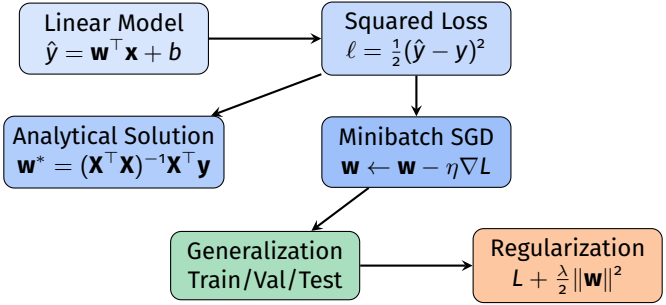


Small gap = Generalizing!

♣ Key Insight

Regularization **increases** training error but **decreases** generalization error. We accept a little bias to reduce variance!

The Big Picture: Everything Connects



Key takeaways:

1. Linear regression **is** a single-layer neural network
2. Squared loss emerges from assuming Gaussian noise (MLE!)
3. Analytical solutions are rare; gradient descent is the workhorse
4. We care about **generalization**, not just training performance
5. Regularization helps when we can't get more data

Key Equations Reference

Concept	Equation
Linear Model	$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$
Matrix Form	$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$
Squared Loss	$\ell^{(i)} = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$
Mean Squared Error	$L = \frac{1}{n}\sum_{i=1}^{n} \ell^{(i)}$
Gradient	$\nabla_{\mathbf{w}} L = \frac{1}{n}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$
Normal Equations	$\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$
OLS Solution	$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$
SGD Update	$\mathbf{w} \leftarrow \mathbf{w} - \eta\nabla_{\mathbf{w}} L$
ℓ_2 Regularization	$L_{\text{reg}} = L + \frac{\lambda}{2}\lVert\mathbf{w}\rVert^2$
Weight Decay Update	$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \eta\nabla_{\mathbf{w}} L$

Exercises to Deepen Your Understanding

1. **Analytical:** Given data $x_1, \ldots, x_n$, find the constant b minimizing $\sum_i (x_i - b)^2$. What is it? (Hint: it's a familiar statistic!)
2. **Implementation:** Initialize weights to zero instead of random. What happens? Why?
3. **Exploration:** Experiment with different learning rates (0.001, 0.01, 0.1, 1.0). Plot the loss curves. What do you observe?
4. **Theoretical:** Why can't we make a "deeper" linear model by stacking two linear layers? (Hint: what's the composition of two linear functions?)
5. **Practical:** For house price prediction, why might $y = \log(\text{price})$ be better than predicting price directly?
6. **Challenge:** Derive the gradient for ℓ_1 regularization ($\sum_i |w_i|$). What's different about this penalty?

What's Next?

You've built the foundation! Next steps:

- **Chapter 4:** Softmax Regression—classification instead of regression
- **Chapter 5:** Multilayer Perceptrons—adding depth!
- **Beyond:** CNNs, RNNs, Transformers...

Thank you!

Full book: `https://d2l.ai`

"The best way to understand deep learning is learning by doing."